

CI/CD for Enterprise Low-Code AI Agents

Low-code accelerates agent creation. CI/CD industrializes agent delivery.

May 2026



MEMBER OF
**Dow Jones
Sustainability Indices**
In Collaboration with RobecoSAM



CI/CD for Enterprise Low-Code AI Agents | Why Maisa Needs CI/CD. The faster we create agents, the more important deployment governance becomes.

Low-code platforms accelerate AI agent creation, but introduce new operational risks:



Manual and inconsistent deployments

Common issues include:

- Different prompt versions between environments
- Missing integrations or permissions in Production
- Undocumented hotfixes performed directly in the platform UI
- Inability to reproduce agent behavior during incidents
- Lack of rollback mechanisms



Limited testing before production

Without a structured testing process, organizations face risks such as:

- Unexpected or inconsistent responses
- Hallucinations or inaccurate outputs
- Failures in edge cases or uncommon scenarios
- Broken integrations with external systems
- Performance degradation after updates



Security and compliance concerns

Key concerns include:

- Unauthorized access to sensitive systems or data
- Excessive permissions granted to agents
- Exposure of credentials, API keys, or secrets
- Lack of auditability for agent changes and decisions
- Non-compliance with regulatory frameworks (e.g., GDPR, internal governance policies)

CI/CD for Enterprise Low-Code AI Agents | Why Maisa Needs CI/CD. The faster we create agents, the more important deployment governance becomes.

R1 Reliability

Consistent behavior across environments

DEV -> PRE -> PRO

R2 Governance

Full change traceability

Everytime an agent goes to PRO, it goes to Governance

R3 Security

Controlled access to integrations and secrets

*Invoke an agent in DEV from an agent in PRO could happen.
Reducing manual deployments reduces human errors.*

R4 Quality

Automated validation before production

Going to PRO without tests

R5 Scalability

Managing dozens or hundreds of agents consistently

Managing one agent is easy...one hundred maybe not

CI/CD for Enterprise Low-Code AI Agents | Why Maisa Needs CI/CD. The faster we create agents, the more important deployment governance becomes.

Challenge / Risk	Business Impact	CI/CD Response	Expected Outcome
Manual & Inconsistent Deployments	Configuration drift across environments, human errors, difficult rollbacks	Version control + automated deployment pipelines (DEV → PRE → PRO)	Consistent and reproducible deployments
Limited Testing Before Production	Unexpected behaviors, hallucinations, broken integrations, performance degradation	Automated validation & evaluation testing before release	Higher reliability and production confidence
Security & Compliance Concerns	Unauthorized access, secret exposure, compliance risks	Approval gates, RBAC, secrets management, auditability	Secure and compliant deployments
Reliability	Different behavior between environments	Controlled environment promotion	Consistent agent behavior
Governance	Lack of traceability	Git versioning + approval workflows	Full change visibility
Quality Assurance	Production incidents due to insufficient testing	CI quality gates	Reduced production failures
Scalability	Hard to manage multiple agents consistently	Standardized delivery model	Scalable agent operations

CI/CD for Enterprise Low-Code AI Agents | Proposed CI/CD Architecture

Develop

Agent Creation & Configuration

Developers, business users, or technical teams design AI agents inside Maisa.

Activities include:

- Creating agents
- Defining prompts and system behavior
- Configuring tools, connectors, and permissions
- Selecting models and runtime parameters

Goal: Rapid and collaborative agent creation.

CI Layer

Automated Validation & Quality Gates

Before promotion, every change is automatically validated.

Validation checks

- Configuration and schema validation
- Prompt quality checks
- Integration verification

Evaluation testing

- Benchmark datasets
- Expected response validation
- Hallucination and consistency testing

Security checks

- Permission validation
- Secrets scanning
- Policy compliance

Maisa Platform

Execution & Business Operations

Once deployed, agents run within Maisa and execute enterprise workflows.

The platform handles:

- User interactions
- API execution
- Business process automation

Governance

Continuous Monitoring & Control

CI/CD does not end after deployment. Organizations continuously monitor:

Technical metrics

- Latency, failures, API health

AI metrics

- Hallucination rate, prompt drift, response consistency

Governance

- Audit trails, approval history, compliance validation

Git Repository

Single Source of Truth

All agent artifacts are version controlled to guarantee reproducibility.

Stored assets include:

- Agent definitions and configurations
- Prompt templates
- Policies and permissions
- Test datasets and evaluation scenarios

CD Layer

Controlled Deployment Across Environments

Validated agents are promoted safely through environments.

DEV → PRE → PROD

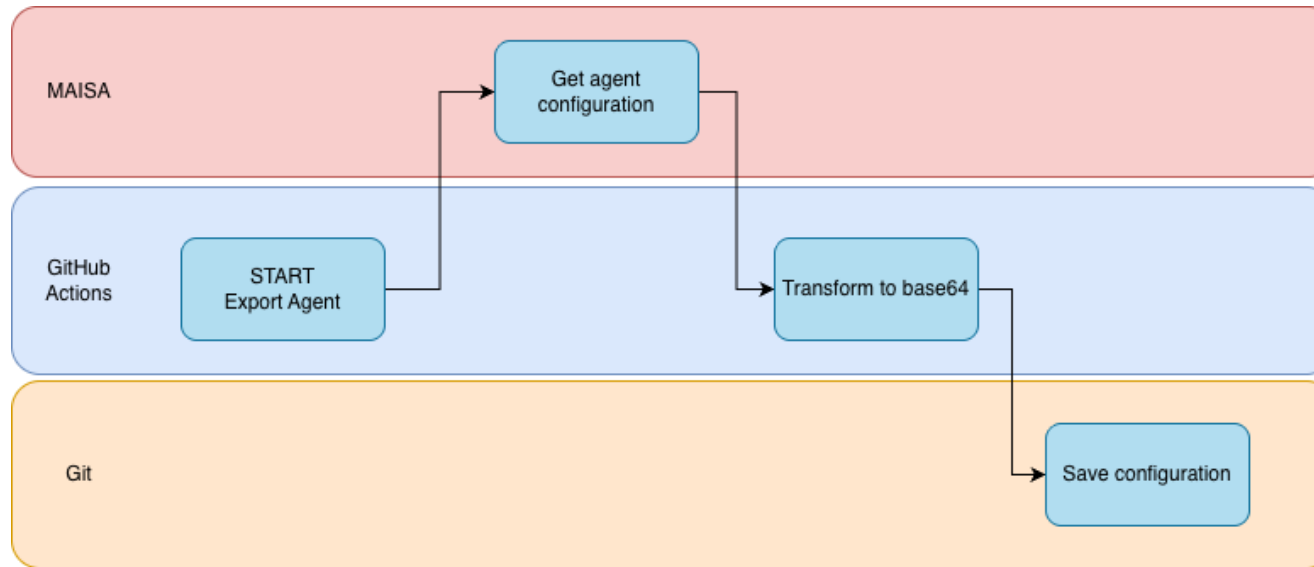
Deployment includes:

- Controlled promotion process
- Approval workflows for production
- Automated rollback when quality drops

CI/CD for Enterprise Low-Code AI Agents | Why Maisa Needs CI/CD. The faster we create agents, the more important deployment governance becomes.

Current Problem

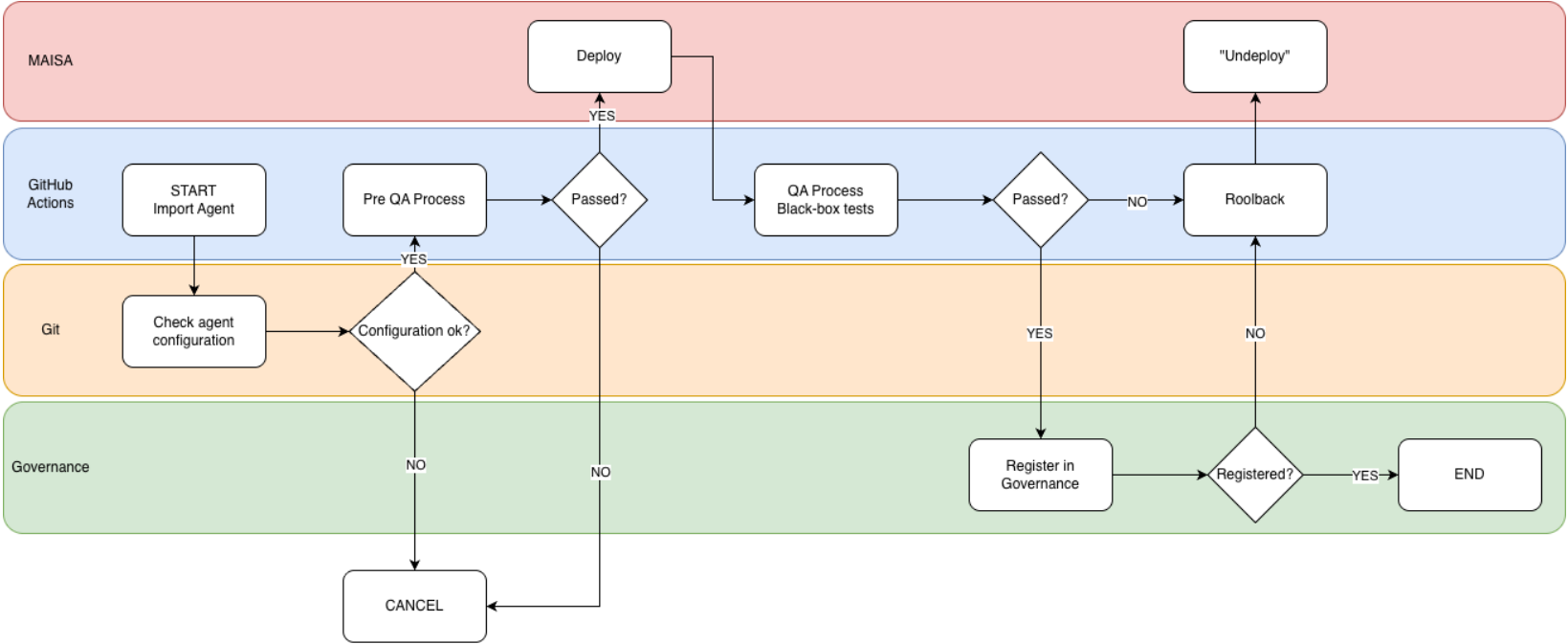
	Current Risk / Impact	CI/CD Solution	Expected Outcome
Unmanaged workers in Production	No visibility of what is deployed in PROD	Version-controlled deployments (Git + release tracking)	Full traceability of deployed workers
Manual deployments	Nobody knows exactly what changed or when	Automated deployment pipelines	Repeatable and auditable deployments
PRE and PROD running different Maisa versions	Workers may fail after deployment due to platform incompatibilities	Environment compatibility validation before release	Safer promotion between environments
No validation of agents / workers / prompts	Errors discovered directly in Production. Differences between different agents with the same prompt.	Automated validation + black-box testing	Fewer production incidents
Ungoverned versioning	Difficult rollback and unclear ownership of changes	Controlled versioning and release management	Reliable rollback and full change history
No controlled movement between environments	Non-audited and unsecured deployments	Approval gates + secure environment promotion	Governed and secure releases
300+ agents cannot be managed manually	Operational bottleneck and high dependency on experts	Standardized CI/CD lifecycle	Scalable agent operations



Export agent

Right now, we are able to export two different things from GitHub Actions:

- `export-config-agents.yml`: Exports the worker manager config JSON (name, status, model, instructions, etc.) in a json file
- `export-agents.yml`: Calls `GET /maisa-bff/workers/{lastVersionId}/export` to export the full agent binary export in a b64 file



Import agent

We have to implement a new workflow to import an agent in a Noxus environment.

We need the import API specification. Right now, we don't have the API detail.

Thank You.

Our purpose is to help people and businesses prosper.

Our culture is based on believing that everything we do should be:

Simple Personal Fair



MEMBER OF
**Dow Jones
Sustainability Indices**
In Collaboration with RobecoSAM



